



TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO DE TUXTEPEC

INGENIERÍA EN SISTEMAS COMPUTACIONALES

ASIGNATURA:

CIENCIA DE DATOS

DOCENTE:

DR. JULIO AGUILAR CARMONA

NOMBRE DEL TRABAJO:

REPORTE "CURSO: AGENTES DE IA CON GOOGLE"

SEMESTRE:

GRUPO:

8°

"A"

NOMBRE DEL ALUMNO:

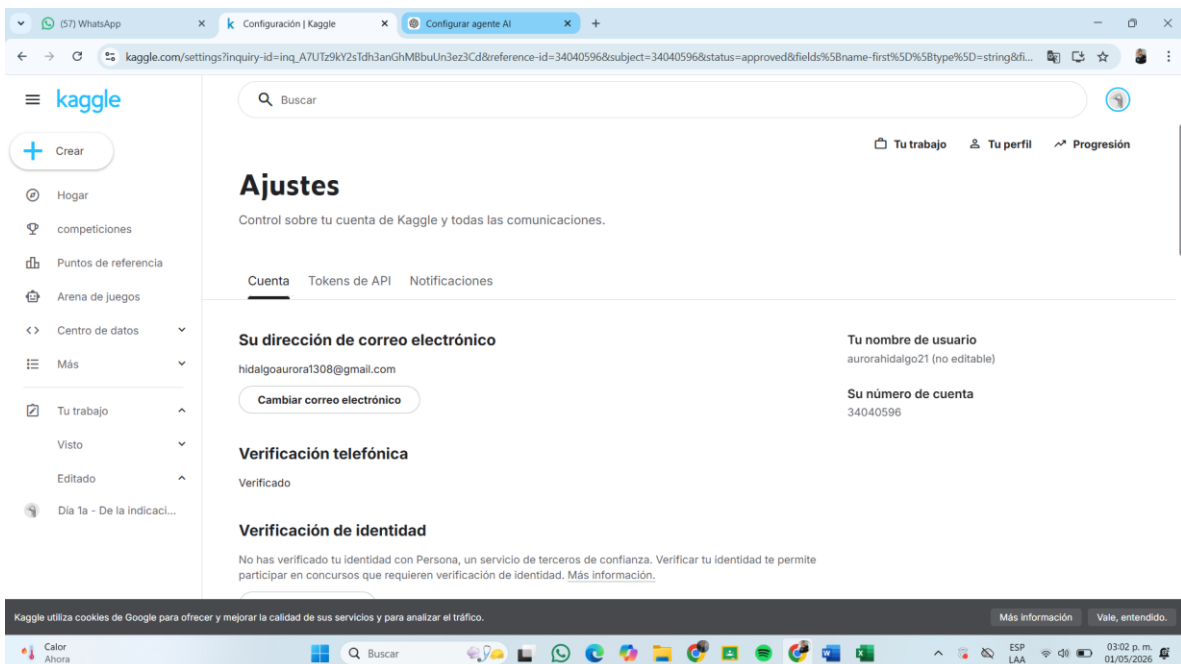
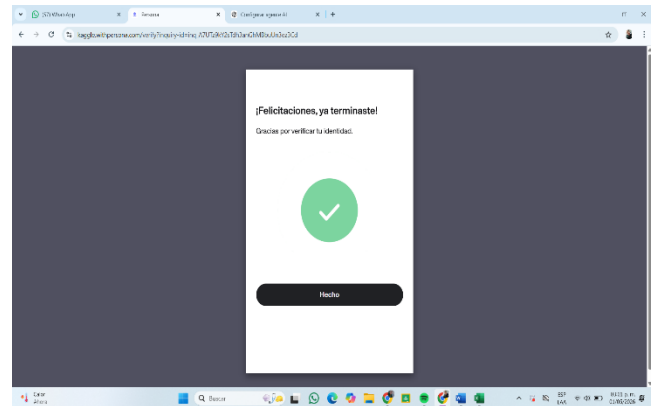
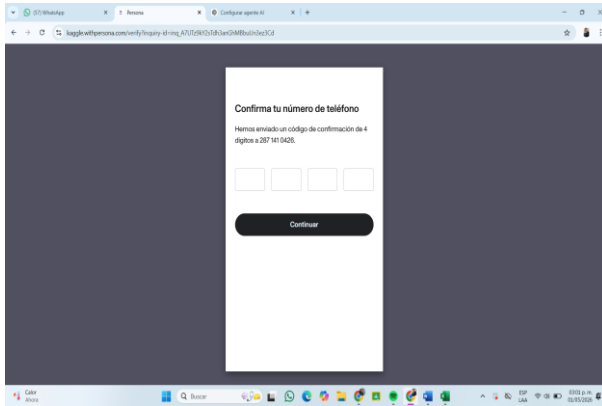
HIDALGO BAJANDO AURORA GUADALUPE

22350711

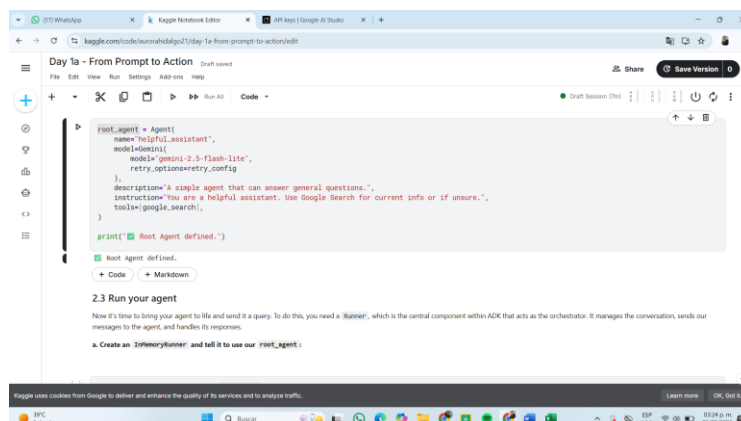
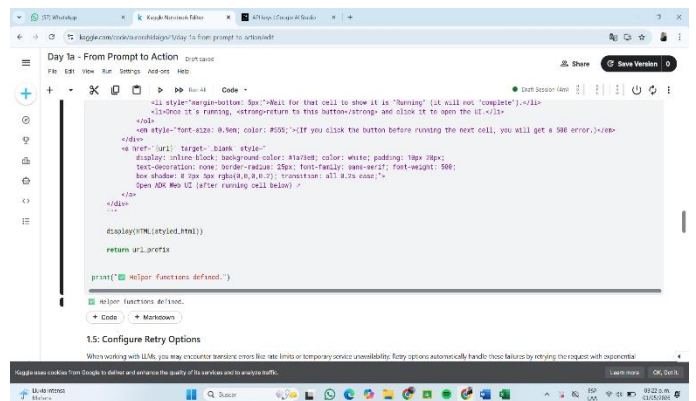
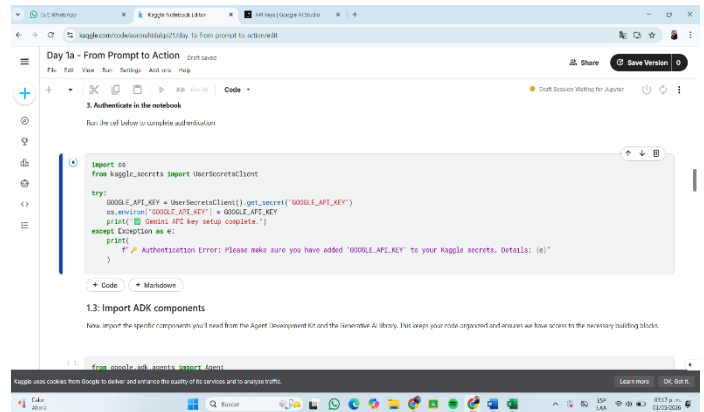
LUNES 27 DE ABRIL DE 2026.

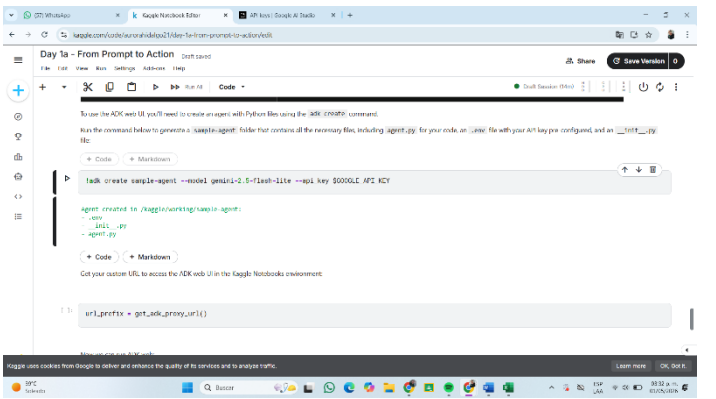
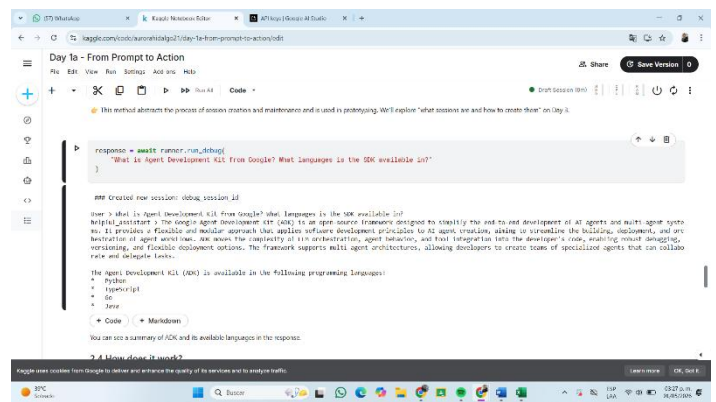
CURSO INTENSIVO: “AGENTES DE IA CON GOOGLE”

Primeramente debemos verificar nuestra cuenta de kaggle con nuestro numero de teléfono para poder dar inicio al curso, y creamos nuestra GOOGLE_API_KEY personal para poder ejecutar los códigos



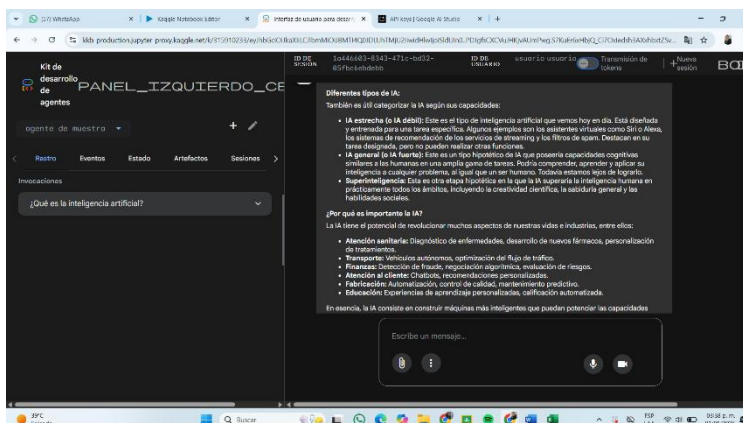
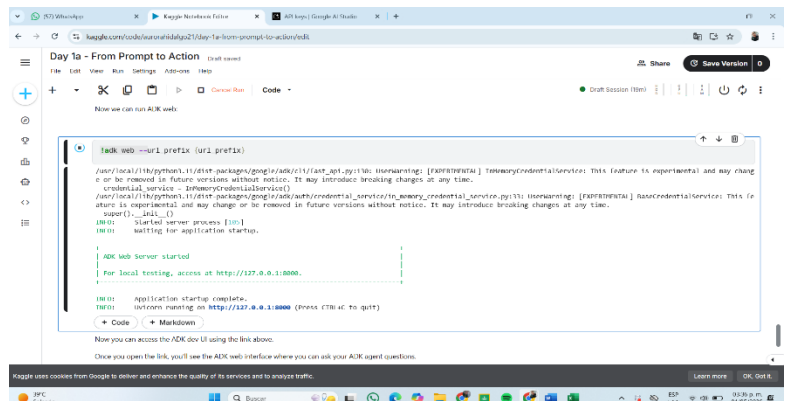
Configuramos el entorno, despues empezamos a Importar librerías de ADK. Luego Creamos un agente básico.

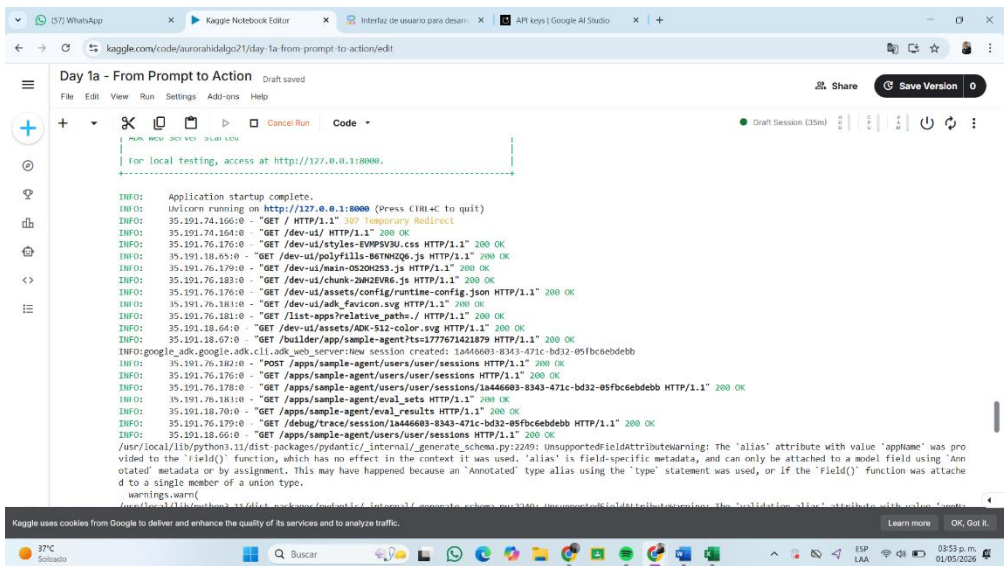




Overview

ADK includes a built-in web interface for interactively chatting with, testing, and debugging your agents.





Uso de la interfaz web ADK Web UI. Y Pruebas de interacción con el agente.

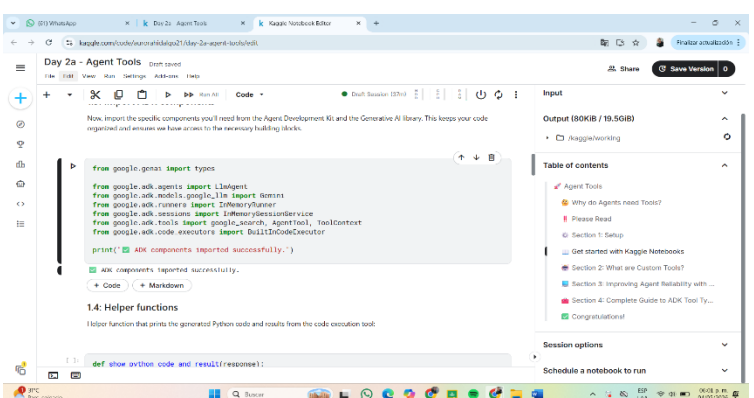
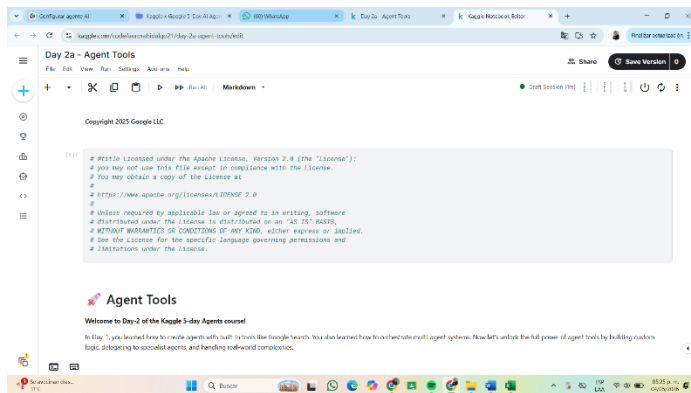
Los aprendizajes obtenidos en el día 1 fueron:

- Comprensión del funcionamiento básico de un agente de IA.
- Uso de modelos Gemini para responder consultas.
- Configuración de entornos de desarrollo para agentes.
- Diferencia entre un modelo conversacional y un agente capaz de ejecutar acciones.

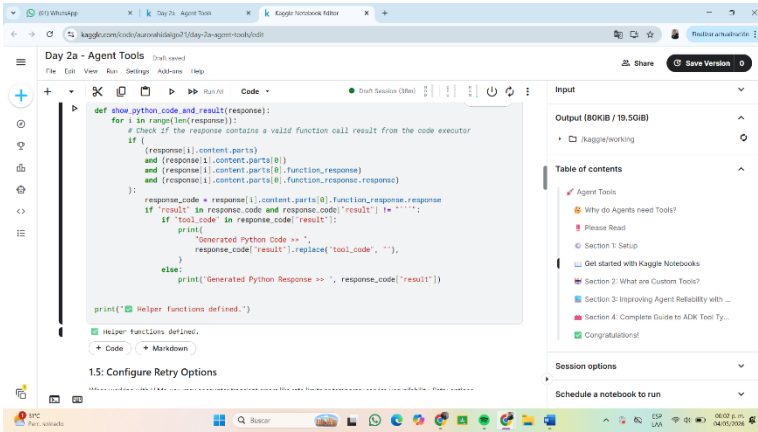
El primer día permitió comprender los fundamentos del desarrollo de agentes inteligentes con Google. Además, se logró configurar correctamente el entorno de trabajo y ejecutar el primer agente funcional utilizando ADK y Gemini.

DIA 2 A – Agent Tools

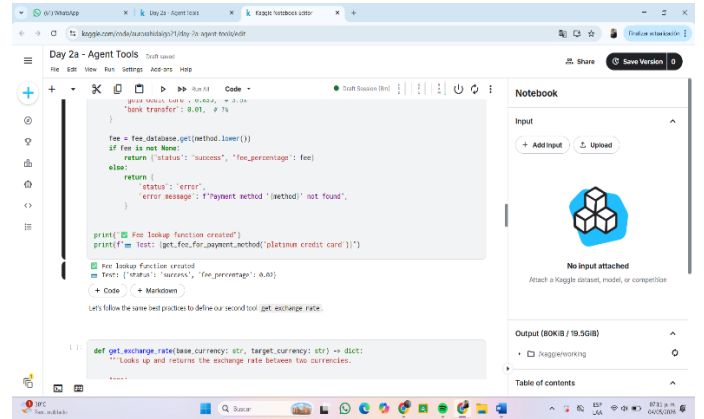
Herramientas para agentes inteligentes



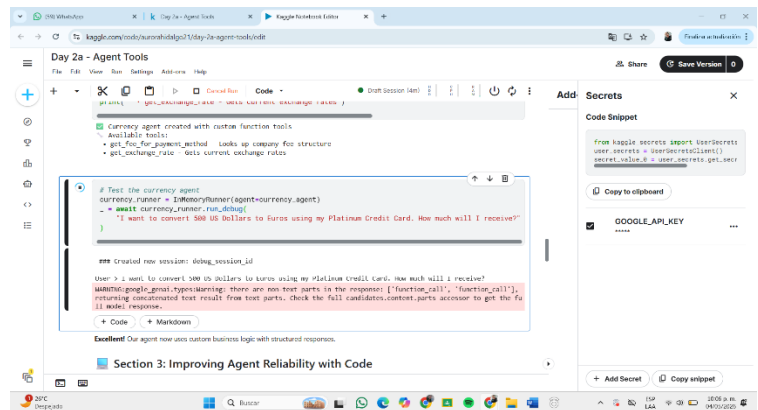
Primeramente en el día 2A Creamosde funciones personalizadas como herramientas. Integración de herramientas dentro de agentes. Uso de múltiples tools en un solo agente.



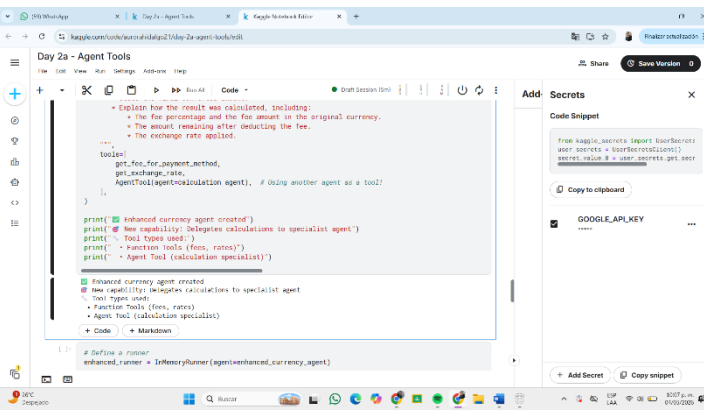
```
def show_python_code_and_result(response):  
    for i in range(len(response)):  
        # Check if the response contains a valid function call result from the code executor  
        if (  
            (response[i].content.parts)  
            and (response[i].content.parts[0])  
            and (response[i].content.parts[0].function_response)  
            and (response[i].content.parts[0].function_response.response)  
        ):  
            response_code = response[i].content.parts[0].function_response.response  
            if "result" in response_code and response_code["result"] != "":  
                if "tool_code" in response_code["result"]:  
                    print("Generated Python Code >> ", response_code["result"])  
                else:  
                    print("Generated Python Response >> ", response_code["result"])  
            else:  
                print("Generated Python Response >> ", response_code["result"])  
        else:  
            print("Generated Python Response >> ", response_code["result"])  
    print("Helper Functions defined.")  
    # Helper functions defined.  
    + Code + Markdown  
1.5: Configure Retry Options
```



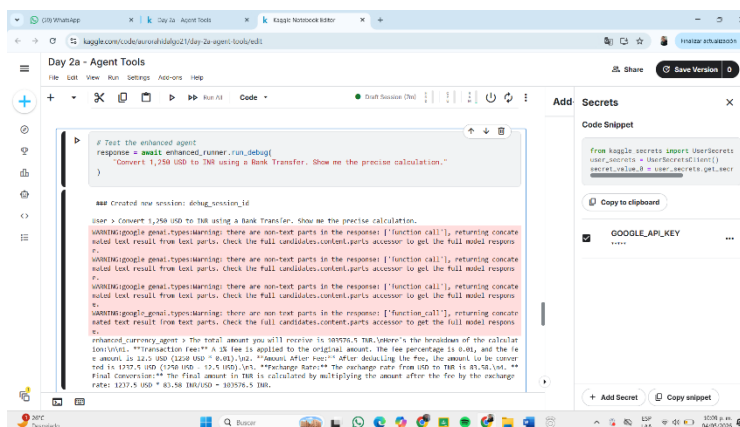
```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:  
    """Looks up and returns the exchange rate between two currencies.  
    """  
    fee = fee_database.get(method_lower())  
    if fee is not None:  
        return {"status": "success", "fee_percentage": fee}  
    else:  
        return {"status": "error",  
                "error_message": f"Payment method '{method}' not found."}  
    print(f"Fee lookup function created")  
    print(f"Fee: {fee}")  
    # Look up and return the exchange rate between two currencies.  
    + Code + Markdown
```



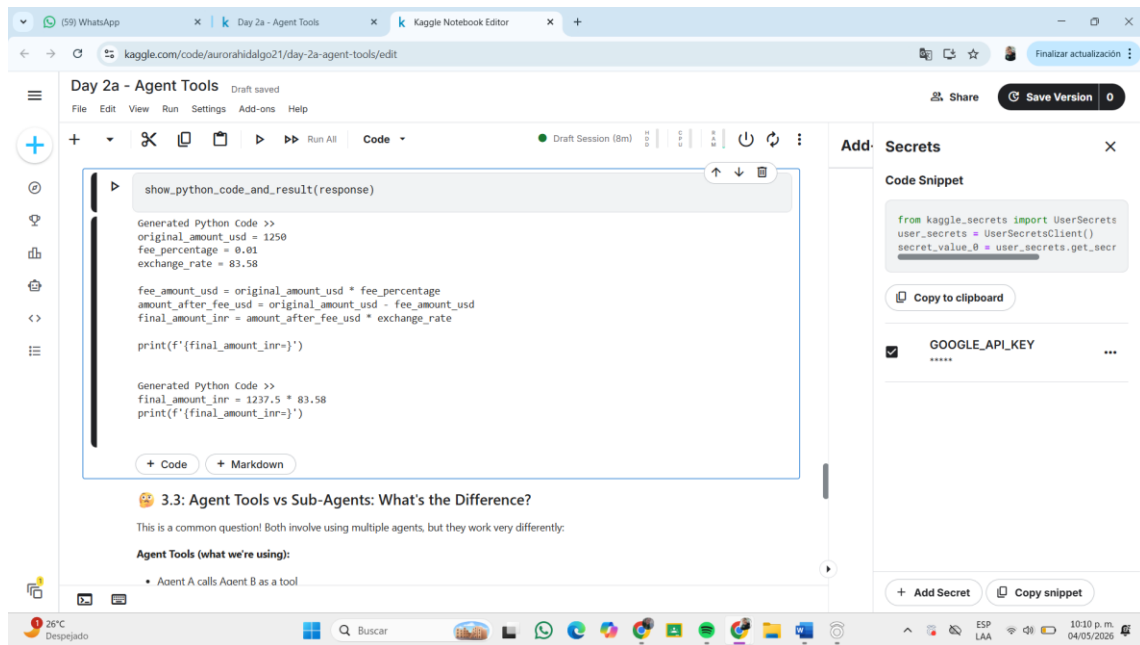
```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:  
    """Looks up and returns the exchange rate between two currencies.  
    """  
    fee = fee_database.get(method_lower())  
    if fee is not None:  
        return {"status": "success", "fee_percentage": fee}  
    else:  
        return {"status": "error",  
                "error_message": f"Payment method '{method}' not found."}  
    print(f"Fee lookup function created")  
    print(f"Fee: {fee}")  
    # Look up and return the exchange rate between two currencies.  
    + Code + Markdown
```



```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:  
    """Looks up and returns the exchange rate between two currencies.  
    """  
    fee = fee_database.get(method_lower())  
    if fee is not None:  
        return {"status": "success", "fee_percentage": fee}  
    else:  
        return {"status": "error",  
                "error_message": f"Payment method '{method}' not found."}  
    print(f"Fee lookup function created")  
    print(f"Fee: {fee}")  
    # Look up and return the exchange rate between two currencies.  
    + Code + Markdown
```



```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:  
    """Looks up and returns the exchange rate between two currencies.  
    """  
    fee = fee_database.get(method_lower())  
    if fee is not None:  
        return {"status": "success", "fee_percentage": fee}  
    else:  
        return {"status": "error",  
                "error_message": f"Payment method '{method}' not found."}  
    print(f"Fee lookup function created")  
    print(f"Fee: {fee}")  
    # Look up and return the exchange rate between two currencies.  
    + Code + Markdown
```



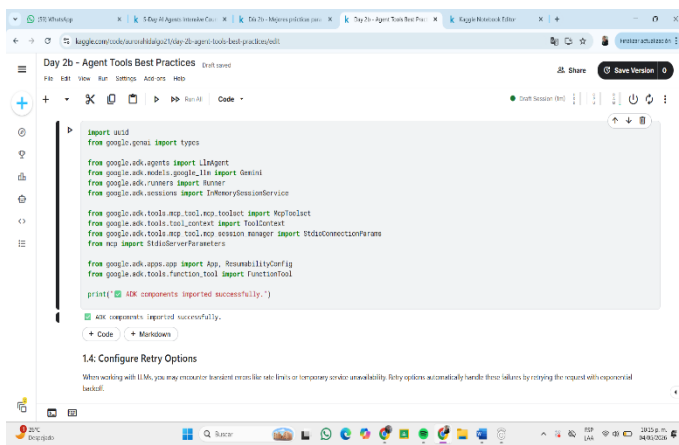
- Simulación de conversiones de divisas.
- Implementación de cálculos automáticos.
- Uso de agentes especializados.

Aprendizajes obtenidos en este día:

- Comprensión del concepto de Agent Tools.
- Uso de funciones Python como herramientas inteligentes.
- Integración de herramientas financieras y de conversión.
- Implementación de agentes multitarea.
- Diferencias entre herramientas y sub-agentes.

DIA 2B – Agent Tools Best Practices

Buenas prácticas para herramientas y workflows



```
import uuid
from google.gemini import types

from google.ai.agents import LLMAgent
from google.ai.agents.tools import Tool
from google.ai.agents.runners import Runner
from google.ai.agents.sessions import InMemorySessionService

from google.ai.agents.tools.mcp.tool_mcp import MCPTool
from google.ai.agents.tools.context import ToolContext
from google.ai.agents.tools.mcp.tool_mcp import MCPToolContext
from google.ai.agents.tools.mcp.tool_mcp import MCPToolContextParameters
from google.ai.agents.tools.mcp.tool_mcp import MCPToolContextParameters

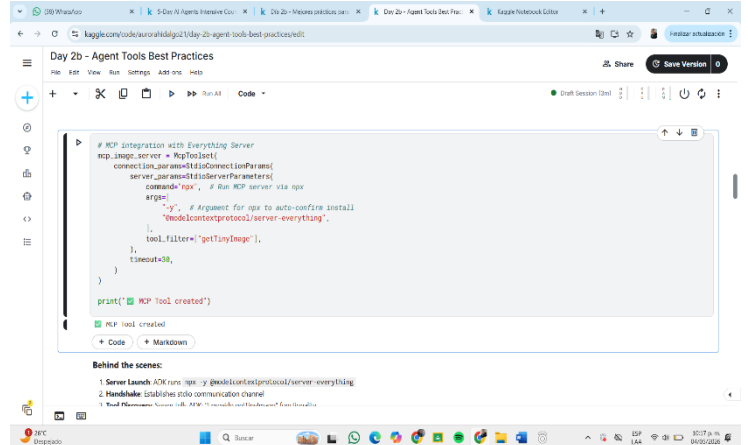
from google.ai.agents.app import Agent, ReusabilityConfig
from google.ai.agents.function import FunctionTool

print('All SDK components imported successfully.')

# SDK components imported successfully.
```

1.4. Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

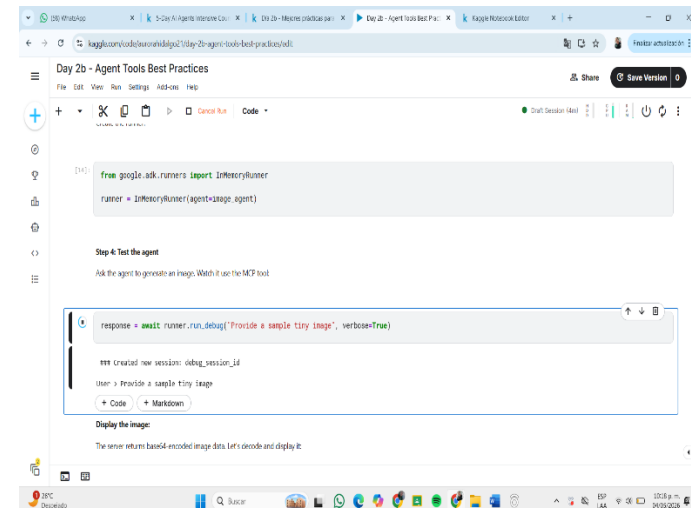


```
# MCP Integration with Everything Server
mcp_image_server = MCPToolContext(
    connection_params=ToolContextParameters(
        server_parameters=ToolContextParameters(
            command_line='python -m mcp_image_server'
        ),
        args=[
            '-y', # Argument for mcp to auto-confirm install
            '--mode=protocol://server-everything',
        ],
        tool_filters=['getTinyImage'],
    ),
    timeout=30,
)

print('MCP Tool created')
```

Behind the scenes:

- Server Launch: MCP server: `python -m mcp_image_server`
- ToolKube: ToolKube is the communication channel
- Tool Parameters: ToolKube is the communication channel



```
from google.ai.agents.runners import InMemoryRunner

runner = InMemoryRunner(agent=agent, agent=agent)
```

Step 4: Test the agent

Ask the agent to generate an image. Watch it use the MCP tool.

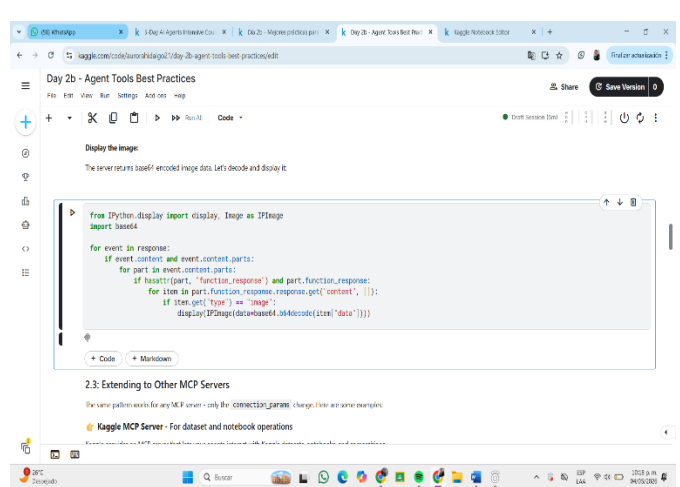
```
response = await runner.run(debug('Provide a sample tiny image', verbose=True))
```

Created new session: debug_session_1d1

User: Provide a sample tiny image

Display the image:

The server returns base64-encoded image data. Let's decode and display it.



Display the image

The server returns base64-encoded image data. Let's decode and display it.

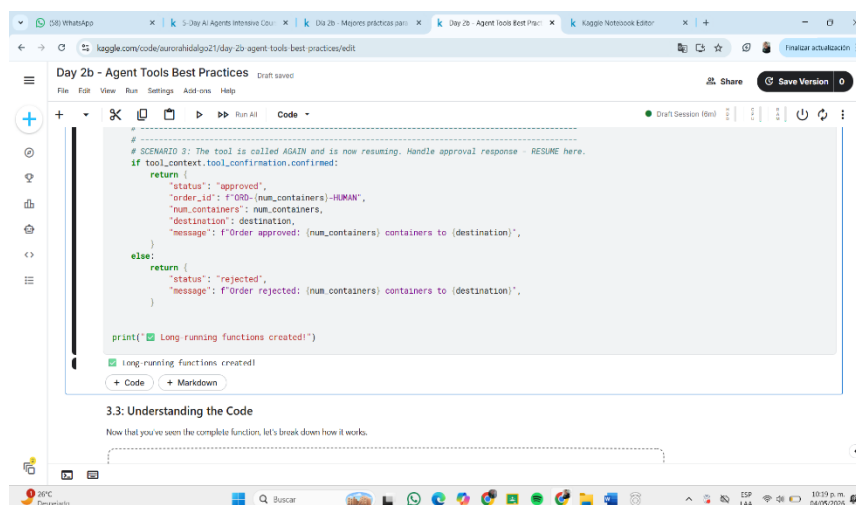
```
from IPython.display import display, Image as IPImage
import base64

for event in response:
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, 'function_response') and part.function_response:
                for item in part.function_response.response.get('content', []):
                    if item.get('type') == 'image':
                        display(IPImage(base64.b64decode(item['data'])))
```

2.3: Extending to Other MCP Servers

The same pattern works for any MCP server - only the `CONNECTION_PARAMS` change. Here are some examples:

- Kaggle MCP Server - For dataset and notebook operations



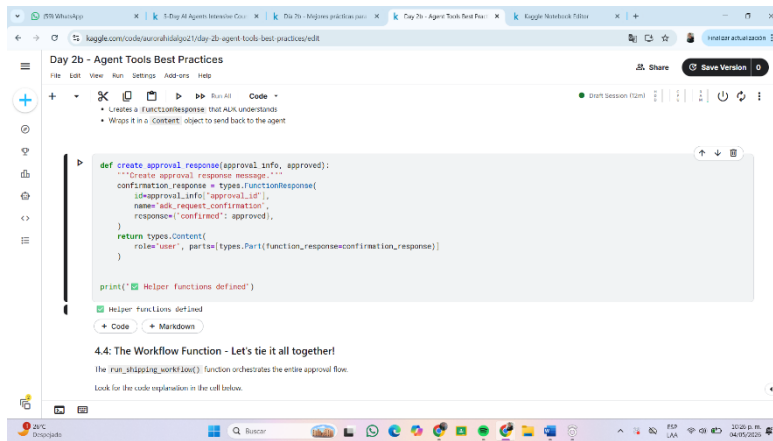
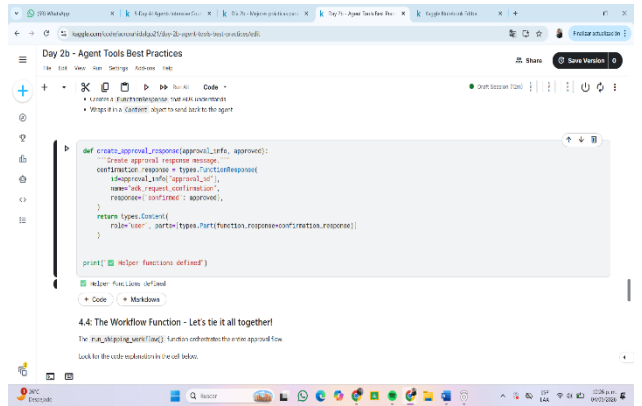
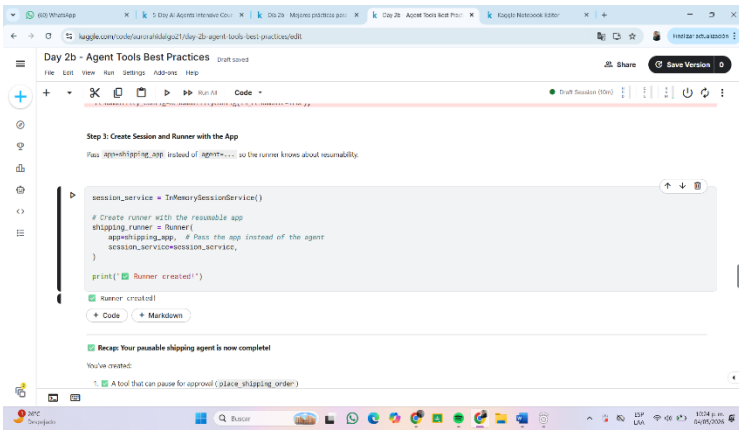
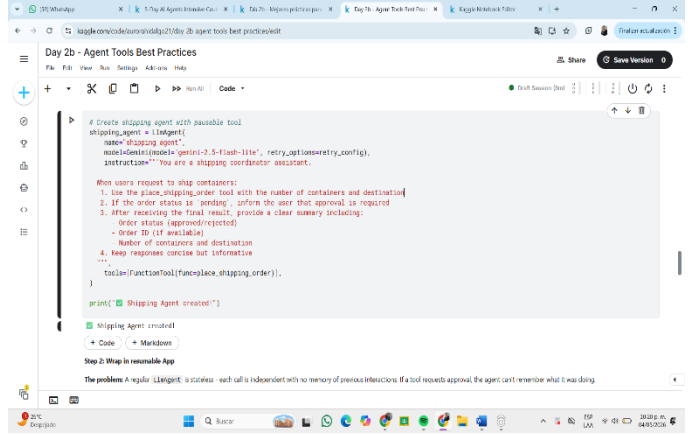
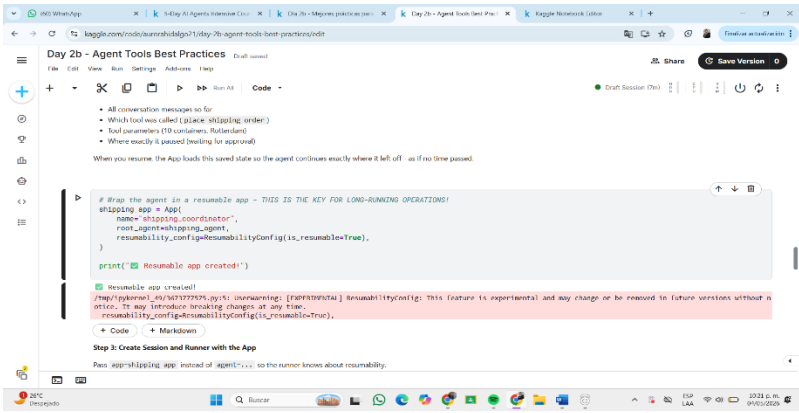
```
# SCENARIO 3: The tool is called AGAIN and is now resuming. Handle approval response - RESUME here.
if tool_context.tool_confirmation.confirmed:
    return {
        'status': 'approved',
        'order_id': f'ORD-{num_containers}-HUMAN',
        'num_containers': num_containers,
        'destination': destination,
        'message': f'Order approved: (num_containers) containers to (destination)',
    }
else:
    return {
        'status': 'rejected',
        'message': f'Order rejected: (num_containers) containers to (destination)',
    }

print('Long running functions created!')
```

Long-running functions created!

3.3: Understanding the Code

Now that you've seen the complete function, let's break down how it works.



```
# Demo 1: It's a small order. Agent receives auto-approved status from tool
await run_shipping_workflow("Ship 3 containers to Singapore")

# Demo 2: Workflow simulates human decision: APPROVE
await run_shipping_workflow("Ship 10 containers to Rotterdam", auto_approve=True)

# Demo 3: Workflow simulates human decision: REJECT
await run_shipping_workflow("Ship 8 containers to Los Angeles", auto_approve=False)

=====
User > Ship 3 containers to Singapore
=====
WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidate.content.parts accessor to get the full model response.
=====
User > Ship 10 containers to Rotterdam
=====
WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidate.content.parts accessor to get the full model response.
/usr/local/lib/python3.11/dist-packages/google/adk/tools/tool_context.py:92: UserWarning: [EXPERIMENTAL] ToolConfirmation: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
ToolConfirmation(
/usr/local/lib/python3.11/dist-packages/google/adk/agents/invoke_context.py:298: UserWarning: [EXPERIMENTAL] BaseAgentState: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
BaseAgentState(
```

Se trabajó principalmente con:

- Workflows de aprobación.
- Operaciones de larga duración.
- Confirmaciones humanas.
- Manejo de estado.
- Reanudación de procesos

ACTIVIDADES REALIZADAS EN ESTE DIA

- Simulación de órdenes de envío.
- Implementación de aprobaciones automáticas.
- Uso de confirmación humana para operaciones críticas.
- Pausa y reanudación de workflows.
- Uso de ToolContext y estados persistentes.

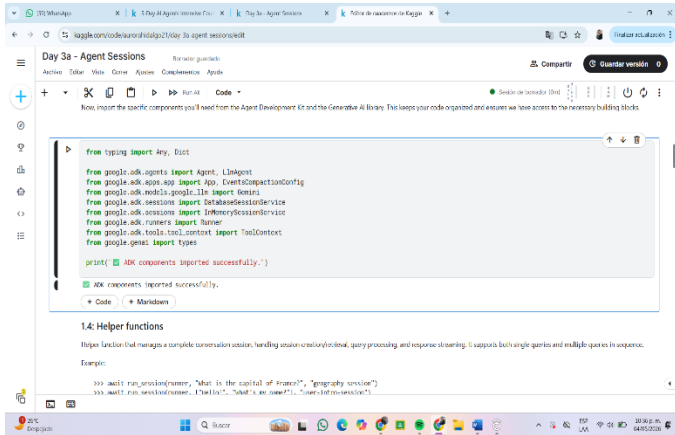
APRENDIZAJE OBTENIDO

- Importancia de las aprobaciones humanas en sistemas inteligentes.
- Manejo de operaciones complejas en agentes.
- Implementación de workflows seguros.
- Reanudación de procesos mediante invocation_id.
- Uso de patrones avanzados de automatización.

DIA 3 Gestión de estado y memoria

Durante el Día 3 se introdujeron conceptos relacionados con el manejo de estado y memoria dentro de los agentes inteligentes.

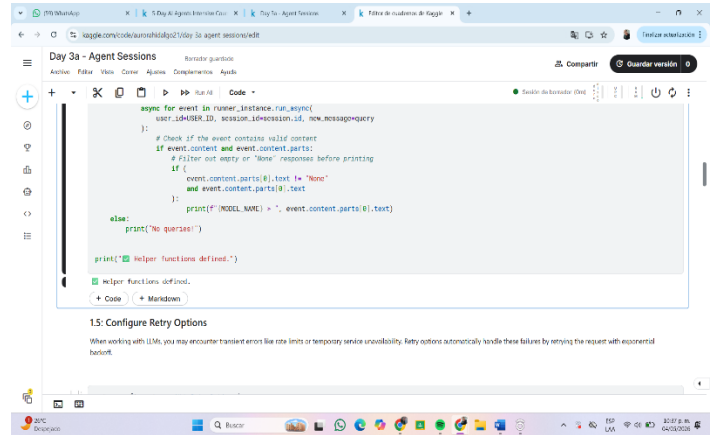
Se analizó cómo los agentes pueden recordar información de conversaciones anteriores y mantener contexto entre múltiples interacciones.



```
from typing import Any, Dict
from google.ai.agents import Agent, LlmAgent
from google.ai.agents.schemas import EventsCompactionConfig
from google.ai.agents.schemas import InMemorySessionService
from google.ai.agents.schemas import InMemorySession
from google.ai.agents.schemas import ToolContext
from google.ai.agents.schemas import ToolContext

print("SDK components imported successfully.")

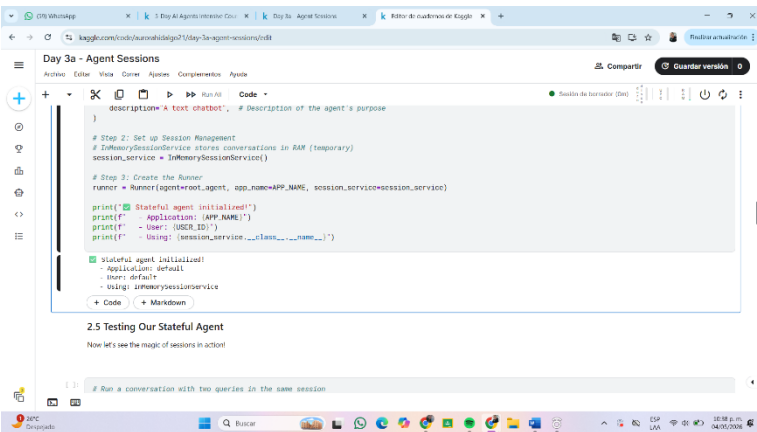
1.4: Helper functions
Helper function that manages a complete conversation session, handling session creation/lookup, query processing, and response streaming. It supports both single queries and multiple queries in sequence.
Example:
>>> await run_session(runner, "What is the capital of France?", "geography service")
>>> await run_session(runner, "What is the capital of France?", "geography service")
```



```
async for event in runner_instance.run_async(
    user_id=USER_ID, session_id=session_id, new_message=query
):
    # Check if the event contains valid content
    if event.content and event.content.parts:
        # Filter out empty or 'None' responses before printing
        if (
            event.content.parts[0].text != "None"
            and event.content.parts[0].text
        ):
            print(f"MODEL NAME: {event.content.parts[0].text}")
        else:
            print("No response!")

print("Helper functions defined.")

1.5: Configure Retry Options
When working with LLMs, you may encounter transient error like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.
```



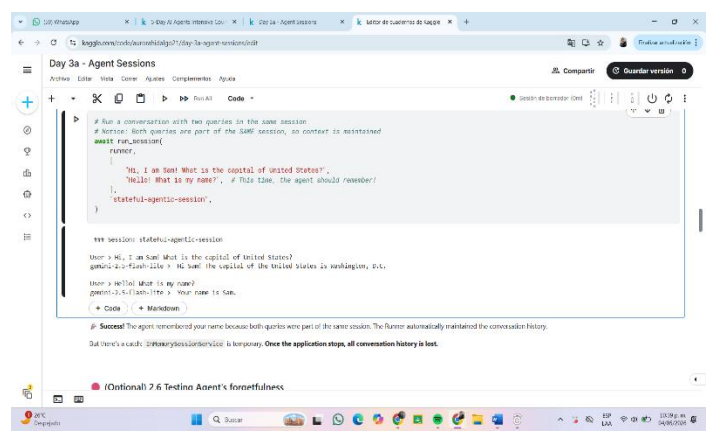
```
description="A text chatbot", # Description of the agent's purpose

# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary)
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=agent, app_name=APP_NAME, session_service=session_service)

print("Stateful agent initialized!")
print(f"Application: {APP_NAME}")
print(f"User: {USER_ID}")
print(f"Session: {session_service._session_id}")

2.5 Testing Our Stateful Agent
Now let's see the magic of sessions in action!
# Run a conversation with two queries in the same session
```

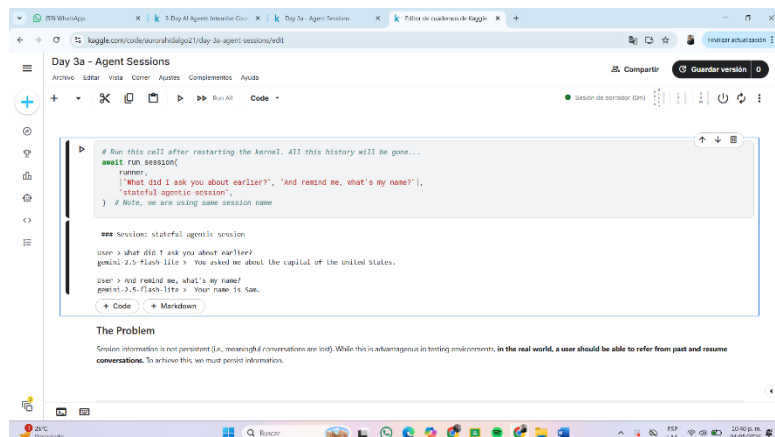


```
# Run a conversation with two queries in the same session
# Note: Both queries are part of the same session, so context is maintained
await run_session(
    runner,
    "Hi, I am Sam! What is the capital of United States?",
    "stateful-agent-session",
    # Note: the agent should remember!
)

# Run the session again
runner = Runner(
    agent=agent, app_name=APP_NAME, session_service=session_service
)

# Note: the agent should remember!
await run_session(
    runner,
    "Hello! What is my name?", # This time, the agent should remember!
    "stateful-agent-session",
)

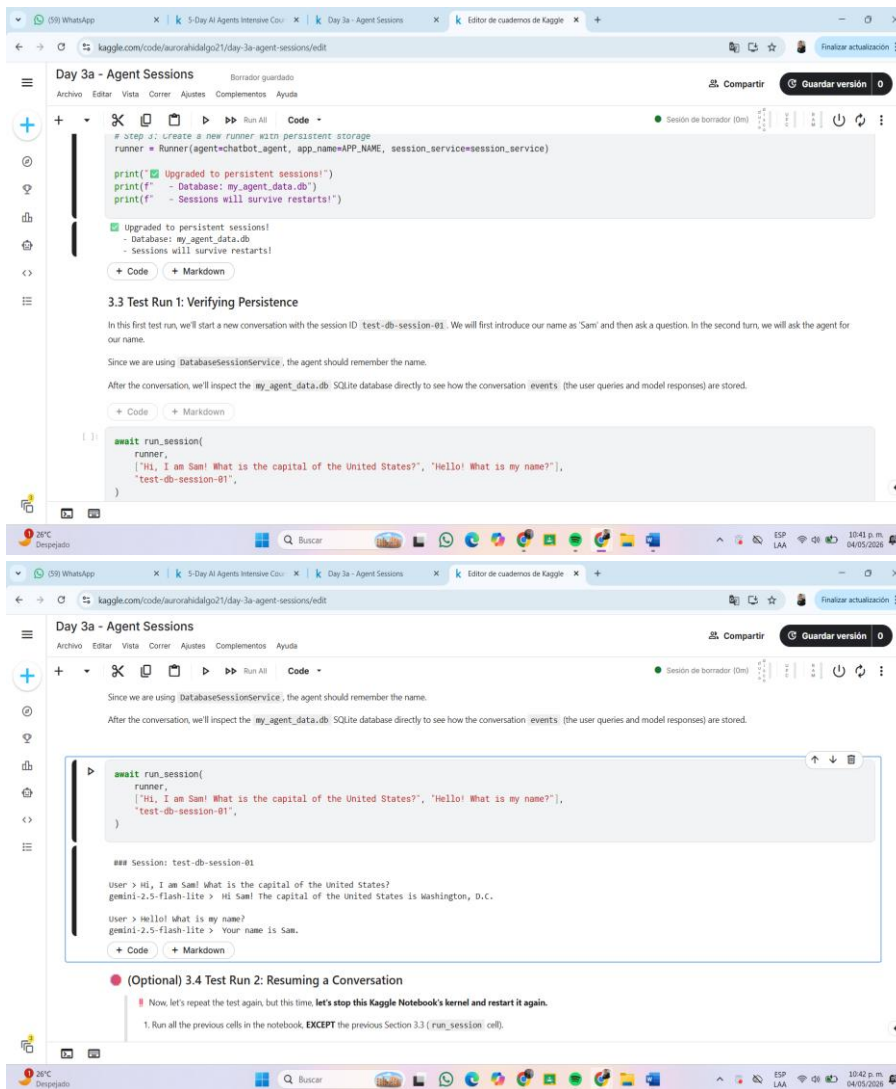
# Success! The agent remembered your name because both queries were part of the same session. The Runner automatically maintained the conversation history.
# But there's a catch: InMemorySessionService is temporary. Once the application stops, all conversation history is lost.
```



```
# Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    "What did I ask you about earlier?", "And remind me, what's my name?",
    "stateful-agent-session",
) # Note, we are using same session name

## Session: stateful-agent-session
user > shut did I ask you about earlier?
gemini-2.0-flash-lite > You asked me about the capital of the United States.
user > And remind me, what's my name?
gemini-2.0-flash-lite > Your name is Sam.

The Problem
Session information is not persistent (i.e., meaningful conversations are lost). While this is advantageous in testing environments, in the real world, a user should be able to refer from past and resume conversations. To achieve this, we must persist information.
```



Aprendizajes obtenidos

- Importancia del manejo de memoria en agentes.
- Diferencia entre sesiones independientes.
- Conservación del contexto conversacional.
- Persistencia de datos entre ejecuciones.